第09课 · 循环嵌套（九九乘法表、图形）

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：打印矩形星号阵（最基础的嵌套，建立信心）

题目：输入两个正整数 n 和 m，打印一个 n 行 m 列的星号矩形。

```
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {        // 外层：控制行数
        for (int j = 1; j <= m; j++) {    // 内层：控制每行的星号个数
            cout << "* ";
        }
        cout << endl;  // 一行打完了，换行
    }
    return 0;
}
```

样例输入：`3 5`

样例输出：

```
* * * * *
* * * * *
* * * * *
```

讲解思路：

1. 先问学生：如果只打印 1 行 5 个星号，怎么写？ 一个 for 就够了。
2. 那如果要打印 3 行呢？每行都是"打印 5 个星号然后换行"，这个动作要**重复 3 次**——外面再套一个循环！
3. 强调 `i` 和 `j` 是两个不同的循环变量，`i` 管行，`j` 管列。就像跑操时，`i` 代表第几个班，`j` 代表班里的第几个同学。

4. **现场演示**：把 `endl` 移到内层循环里面，运行看看会变成什么样——所有星号都变成一列了！帮学生理解 `endl` 位置的重要性。

例题 2：打印直角三角形（内层终点依赖外层变量）

题目：输入正整数 n，打印一个 n 行的直角三角形。

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {       // 第 i 行
        for (int j = 1; j <= i; j++) {   // 第 i 行打印 i 个星号
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

样例输入：`5`

样例输出：

```
*
* *
* * *
* * * *
* * * * *
```

讲解思路：

1. **和例题 1 对比**：唯一的变化是内层循环的终点从 `m`（固定值）变成了 `i`（跟着外层变）！
2. **生活比喻：叠积木**

   > 想象你在叠积木塔：第 1 层放 1 块，第 2 层放 2 块，第 3 层放 3 块……第 i 层放 i 块。
   > 每一层放多少块，取决于你现在是第几层——**内层循环的次数由外层的 `i` 决定**。

3. 手动模拟前 3 行：

外层 i	内层 j 的范围	打印星号个数	输出
1	j: 11	1 个	`*`
2	j: 12	2 个	`* *`
3	j: 13	3 个	`* * *`

1. 关键提问：第 4 行打印几个星号？（4 个！）最后一行（第 n 行）打印几个？（n 个！）

例题 3：九九乘法表（经典中的经典）

题目：打印九九乘法表。

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 9; i++) {           // i: 行号，也是第一个乘数
        for (int j = 1; j <= i; j++) {       // j: 列号，也是第二个乘数
            cout << j << "*" << i << "=" << i * j << "\t";
        }
        cout << endl;
    }
    return 0;
}
```

样例输出（部分）：

```
1*1=1
1*2=2    2*2=4
1*3=3    2*3=6    3*3=9
1*4=4    2*4=8    3*4=12    4*4=16
...
1*9=9    2*9=18    3*9=27    4*9=36    5*9=45    6*9=54    7*9=63    8*9=72    9*9=81
```

讲解思路：

1. **先建立直觉**：在黑板上写出九九乘法表的前 3 行，问学生——

   > 看，第 1 行只有 1 个算式，第 2 行有 2 个，第 3 行有 3 个……第 i 行有 i 个！是不是和刚才的三角形一样？

2. **类比三角形**：三角形每行打印 `i` 个星号，乘法表每行打印 `i` 个算式。框架完全一样，只是循环体里"打印什么"不同。

3. **内容拆解**：每个算式的格式是 `j * i = i*j`。

> `j` 是小的那个数（列号），`i` 是大的那个数（行号），结果是 `i * j`。

4. **新知识：`\t`（制表符）**：

> `\t` 就像按一下键盘上的 Tab 键，能让输出**自动对齐**。如果不用 `\t`，乘法表会挤在一起，很难看。

5. **现场互动**：先让学生猜第 5 行有几个算式（5 个），最后一个算式是什么（`5*5=25`）。

例题 4：打印倒直角三角形（反向思考）

题目：输入正整数 n，打印一个倒直角三角形。

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = n; i >= 1; i--) {        // 从第 n 行开始，倒着来
        for (int j = 1; j <= i; j++) {    // 第 i 行打印 i 个星号
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

样例输入：`5`

样例输出：

```
* * * * *
* * * *
* * *
* *
*
```

讲解思路：

1. **和例题 2 对比**：正三角形是 `i` 从 1 增长到 n，倒三角形是 `i` 从 n 减小到 1。
2. 第 7 课学过"倒数"（`i--`），现在用到嵌套里了——**旧知识在新场景中复用**。

3. 还有另一种思路：外层 `i` 仍然从 1 到 n，但内层终点写成 `n - i + 1`。两种都可以，现场对比一下。
4. **成就感设计**：让学生自己先试，能写出来的同学举手，老师记名表扬。

附：例题源代码与运行结果

例题 1：Print a Rectangle of Stars (Basic Nesting)

```
// Input n (rows) and m (columns), print an n x m rectangle of stars.
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {       // Outer: controls rows
        for (int j = 1; j <= m; j++) {   // Inner: controls columns per row
            cout << "* ";
        }
        cout << endl;  // Newline after each row
    }
    return 0;
}
```

Sample Input

```
3 5
```

Expected Output

```
* * * * *
* * * * *
* * * * *
```

例题 2：Right Triangle of Stars (Inner Depends on Outer)

```
// Input a positive integer n, print a right triangle with n rows.
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {       // Row i
        for (int j = 1; j <= i; j++) {   // Print i stars on row i
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Sample Input

```
5
```

Expected Output

```
*
* *
* * *
* * * *
* * * * *
```

例题 3：Multiplication Table (Classic Nested Loop)

```
// Print the 9x9 multiplication table.
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 9; i++) {           // i: row number (first factor)
        for (int j = 1; j <= i; j++) {       // j: column number (second factor)
            cout << j << "*" << i << "=" << i * j << "\t";
        }
        cout << endl;
    }
    return 0;
}
```

Expected Output

```
1*1=1
1*2=2    2*2=4
1*3=3    2*3=6    3*3=9
1*4=4    2*4=8    3*4=12    4*4=16
1*5=5    2*5=10    3*5=15    4*5=20    5*5=25
1*6=6    2*6=12    3*6=18    4*6=24    5*6=30    6*6=36
1*7=7    2*7=14    3*7=21    4*7=28    5*7=35    6*7=42    7*7=49
1*8=8    2*8=16    3*8=24    4*8=32    5*8=40    6*8=48    7*8=56    8*8=64
1*9=9    2*9=18    3*9=27    4*9=36    5*9=45    6*9=54    7*9=63    8*9=72    9*9=81
```

例题 4：Inverted Right Triangle (Reverse Thinking)

```
// Input a positive integer n, print an inverted right triangle.
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = n; i >= 1; i--) {       // Start from row n, go down to 1
        for (int j = 1; j <= i; j++) {   // Print i stars on current row
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Sample Input

```
5
```

Expected Output

```
* * * * *
* * * *
* * *
* *
*
*
```